Global Partitions for Boosting Time Series Classification

Matthew Yang

October 17, 2026

Abstract

Gradient boosted decision trees are the usual classifier to put on top of interval-based time series features, but inducing each tree means searching for splits over the whole training set. We replace the tree with a *global partition*: b random threshold predicates, chosen from pairs of hard, differently labelled examples. The predicates are applied to every example at once and index a table of 2^b leaf values. The resulting booster, HashBoost, does no split search. It trains from mini-batches, so its memory use does not depend on the size of the training set. We implement it on the GPU in PyTorch and evaluate it after a QUANT transform on five MONSTER datasets and on the 112 standard UCR datasets. On UCR, default HashBoost is statistically level with default XGBoost (mean ranks 2.36 and 2.39) and clearly behind QUANT's own ExtraTrees classifier (1.25). It beats XGBoost on small training sets and trails both on the largest. Fewer bits per round and more rounds, chosen by cross-validation over datasets, cut its mean error from 0.196 to 0.165. That is significantly ahead of XGBoost (0.210) and still behind ExtraTrees (0.148). On all 26 MONSTER datasets, with each model's configuration chosen by the same procedure on validation data and frozen before testing, HashBoost is level with tuned XGBoost on MONSTER's five test folds (error ratio 1.02, Wilcoxon $p = 0.98$) and trains 8.2× faster in total on the same GPU. On MONSTER's larger training sets both boosters rank significantly ahead of ExtraTrees, the reverse of UCR. Streaming the 975,291-row LenDB training pool, HashBoost's peak GPU memory is the same as at 65,536 rows (2.6 GB), while XGBoost's binned matrix grows linearly to 14.6 GB. HashBoost's limit is the number of rounds it can afford, not memory: every batch updates every existing round, so training cost is quadratic in rounds. Freezing old rounds makes it linear with little loss of accuracy.

Contents

1 Introduction **3**

2 Background and Related Work **3**

3 Method **4**
3.1 A round is a global partition 4
3.2 Choosing the predicates 5
3.3 Fitting the leaves 5
3.4 Extensions 5
3.5 Implementation 6

4 Experimental Setup **7**

5 Results **10**
5.1 MONSTER: the five test folds (RQ1) 10
5.2 Development on five datasets 12
5.3 The UCR archive (RQ3) 14
5.4 Training cost and scaling with instances (RQ2) 16

5.5 What did not work 18
5.6 Run-to-run noise 19

6 Discussion **20**

7 Conclusion **21**

A Remaining work before submission **22**

1 Introduction

Gradient boosted models are built from a sequence of decision trees, where each tree is fitted to the residual errors of the ensemble constructed so far rather than to the original targets [1]. This stage-wise additive formulation, together with highly engineered implementations such as XGBoost [2], LightGBM [3] and CatBoost [4], has made boosting the default choice for tabular prediction tasks, where it remains competitive with, and frequently superior to, deep learning [5, 6].

Boosted trees are also the usual classifier on top of interval-based time series features. QUANT [7] passes its quantile features to an ensemble of extremely randomised trees, and the features produced by PULSAR [8] and its predecessors are used the same way. At the scale of the MONSTER benchmark [9], with up to 1.5 million series per dataset, the classifier becomes the expensive part. Inducing a tree requires evaluating candidate split points at every node, so each boosting iteration costs time proportional to both the number of instances and the number of features. Histogram binning, approximate split finding, gradient-based sampling and feature bundling [2, 3, 10] all reduce the constant factor. They do not remove the need to revisit the whole training set in every round. In practice this also means holding it in memory: XGBoost's binned representation of LenDB after QUANT is 14.6 GB (Section 5.4).

This paper asks whether the tree is needed at all. We replace each tree with a *global partition* of the feature space: b axis-aligned threshold predicates, applied to every example at once. The resulting b bits index a table of 2^b leaf values. The predicates are not searched for. Each one takes a random feature and puts its threshold at the midpoint of two hard, differently labelled examples. Each leaf value is a Newton step computed from running sums of the gradient and hessian over the examples that land in it. Because the partition does not depend on which examples are in the current batch, those running sums can be updated from any later mini-batch. The model never needs the training set in one place.

Contributions.

1. **HashBoost**, a gradient booster whose weak learner is a random global partition, and a GPU implementation that is 9.6× faster than the original CPU implementation and agrees with it to 4×10^{-7} (Sections 3 and 3.5).

2. Two changes that make it scale. *Bagging* divides the quadratic cost in rounds by the number of estimators. *Round freezing* with a per-row cache turns the quadratic cost into a linear one (Section 3.4).

3. An evaluation on all 26 MONSTER datasets, trained on whole pools of up to 4.65 million rows and scored on MONSTER's five test folds against XGBoost tuned by the same procedure and against QUANT's ExtraTrees (Section 5.1). Development runs on five MONSTER datasets also compare LightGBM and CatBoost (Section 5.2). On the 112 standard UCR datasets we compare against QUANT's classifier and XGBoost, with Friedman and Nemenyi tests (Section 5.3).

4. A set of negative results. Making individual partitions smarter (gain-based selection, feature weighting, richer features) usually makes the ensemble *worse*, and run-to-run noise is larger than seeding suggests (Sections 5.5 and 5.6).

2 Background and Related Work

Boosting. Boosting originates in the observation that a set of weak learners can be combined into an arbitrarily strong one, formalised by AdaBoost [11] and later generalised to arbitrary differentiable losses as gradient boosting [1]. Decision trees are the conventional weak learner,

but nothing in the gradient boosting framework requires them. Modern implementations fit each tree to a second-order expansion of the loss, so that a leaf's value is a Newton step $-\sum g / \sum h$ over the examples it contains [2]. HashBoost keeps this leaf rule and changes how examples are assigned to leaves.

Oblivious trees and decision tables. CatBoost [4] grows *oblivious* trees, in which every node at a given depth uses the same split. A depth-d oblivious tree is therefore d predicates whose outcomes index 2^d leaves. This is the same structure as one HashBoost round, and boosted decision tables [12] and neural oblivious ensembles [13] use it too. The differences are in how the predicates are chosen and when the leaves are fitted. CatBoost chooses each level's split greedily by gain over the whole training set, and fixes the leaves once the tree is built. HashBoost chooses predicates at random from a single mini-batch, and keeps re-estimating every round's leaves as later batches arrive (Section 3). Boosted decision tables [12] are the closest relative. They choose a table's tests greedily by squared-error gain over the whole training set, then *backfit* them: each test is removed and re-chosen given the others, one pass at a time. A table's leaf values are set once, as the mean target in each partition, and earlier tables are never revisited. HashBoost does the opposite on both counts. Its tests are never searched for, and the leaves of every earlier round are re-estimated by each later batch. That online re-estimation of leaves across rounds is closer to backfitting additive models [14] than to BDT's backfitting of splits within one table.

Randomised partitions. Random forests [15] and extremely randomised trees [16] showed that much of the split search in a tree can be replaced by randomness without losing accuracy, provided the ensemble is large. Extremely randomised trees draw both the feature and the threshold at random, and QUANT uses them as its classifier [7]. A random global partition can also be read as a locality-sensitive hash [17]: examples that share a code agree on b random comparisons. HashBoost's thresholds are random but not uninformed. They sit between two hard examples of different classes, so each bit tends to separate examples the current model confuses.

Time series classification. In time series classification, random-feature methods have shown that the expensive, data-dependent search performed by tree induction can often be replaced by cheap, data-independent random transformations. ROCKET [18] applies a large bank of random convolutional kernels followed by a linear classifier. It achieves accuracy comparable to far more expensive ensembles such as HIVE-COTE [19] at a small fraction of the training cost. MiniRocket [20] and Hydra [21] refine this approach further. A parallel line of work extracts features from intervals of the series. Time Series Forest [22] computes simple summary statistics over randomly selected intervals and passes them to a tree ensemble. The idea was developed further in CIF [23], DrCIF [19], rSTSF [24], QUANT [7] and PULSAR [8]. All of these end in some form of tree. We use QUANT and PULSAR as fixed feature transforms and replace only the classifier.

3 Method

3.1 A round is a global partition

Let $x \in \mathbb{R}^p$ be a feature vector and k the number of classes. A round r is defined by b pairs $(j_{r,i}, t_{r,i})$ of a feature index and a threshold, for $i = 0, \ldots, b-1$. It maps x to a b-bit code

$$c_r(x) = \sum_{i=0}^{b-1} 2^i \mathbb{1}\left[x_{j_{r,i}} \le t_{r,i}\right] \in \{0, \ldots, 2^b - 1\}, \tag{1}$$

and has a table $L_r \in \mathbb{R}^{2^b \times k}$ of leaf values. The model after R rounds is

$$F(x) = \sum_{r=1}^{R} L_r[c_r(x)] \in \mathbb{R}^k, \tag{2}$$

and predicts softmax($F(x)$). We use $b = 8$ throughout, so each round has 256 buckets. A round is an oblivious tree of depth 8 with every split fixed in advance.

3.2 Choosing the predicates

Given the current batch, its probabilities and its labels, a new round's predicates are chosen as follows (`HardPairSplitter`):

1. Order the batch by cross entropy, hardest first.
2. Walk that order and greedily form b pairs of examples with different labels.
3. For each pair (u, v), draw a feature j uniformly at random and set $t = (u_j + v_j)/2$.

Each bit therefore separates two examples the model currently gets wrong, and the only search is the sort in the first step. The *sampled* variant replaces the strict ordering with successive draws without replacement, proportional to cross entropy, using the Gumbel-top-k trick [25]. The strict ordering keeps pairing the same dozen persistent outliers each time a batch comes round, and sampling spreads the pairs across the hard end of the batch.

3.3 Fitting the leaves

We use the softmax cross-entropy gradient $g = p - \text{onehot}(y)$ and a floored diagonal hessian $h = p(1 - p) + 10^{-3}$. Every round keeps running sums $G_r, H_r \in \mathbb{R}^{2^b \times k}$ of $-g$ and h over the examples that landed in each bucket, and its leaf values are

$$L_r[s] = \eta \, \frac{G_r[s]}{H_r[s] + \epsilon}, \tag{3}$$

with learning rate $\eta = 0.1$ and $\epsilon = 10^{-6}$.

The key departure from tree boosting is *when* these sums are accumulated. A tree's leaves are fitted once, when the tree is built, and never change. In HashBoost, **every mini-batch is accumulated into every existing round**, with gradients evaluated at the current model (Algorithm 1). A round created early keeps absorbing evidence from every batch that follows. This is what lets the model learn from a stream, since a round is not tied to the batch that created it. It resembles backfitting of additive models [14], applied online. It is also where the cost comes from: a batch costs $O(R)$ once there are R rounds, so T batches with one new round each cost $O(T^2)$.

3.4 Extensions

All of the following are opt-in. The defaults reproduce the algorithm above, which is checked against an independent reference implementation (Section 3.5).

Capacity (H rounds per batch). `hashes_per_round` $= H$ adds H rounds per batch instead of one, which raises capacity without more passes over the data. Because each of the H steps re-accumulates the batch into the existing rounds, it also weights each batch H times in old rounds. A variant that added two rounds but updated old rounds only once lost most of the gain on Pedestrian (0.2301 against 0.2122).

Algorithm 1 One HashBoost step on a mini-batch (X, y) of n examples. The default adds $H = 1$ round per batch.

for $h = 1, \ldots, H$ **do**
  $F \leftarrow \sum_{r<R} L_r[c_r(X)]$ ▷ (n, k) logits
  $p \leftarrow \mathrm{softmax}(F)$; $g \leftarrow p - \mathrm{onehot}(y)$; $h \leftarrow p(1-p) + 10^{-3}$
  choose predicates of round R from hard pairs in (X, y, p)
  for $r = 0, \ldots, R$ **do** ▷ all rounds, including the new one
    $G_r[c_r(X)]$ += $-g$; $H_r[c_r(X)]$ += h ▷ scatter-add
    $L_r \leftarrow \eta\, G_r/(H_r + \epsilon)$
  end for
  $R \leftarrow R + 1$
end for

Bagging. `BaggedHashBoost` averages the logits of E independent models, each trained on the full data with its own random draws. Cost is quadratic in rounds *per model*, so E models of R/E rounds cost R^2/E against R^2 for one deep model.

Round freezing. With `active_rounds` $= W$, only the newest W rounds keep accumulating. Older rounds are frozen, and a frozen round's contribution to a given example never changes again. Given stable row ids, each row's frozen contribution is summed once, cached in an (N, k) table, and never recomputed. A batch then costs $O(W + f)$, where f is the number of rounds frozen since its rows were last seen, so the total cost becomes linear in rounds. In experiments W is set in epochs (`frozen_10ep` keeps ten epochs' worth of rounds active).

Refitted readout. Each leaf is a Newton step computed as if every other round were fixed. After boosting, the tables can instead be fitted *jointly*. L has shape $(R, 2^b, k)$, the same as a multiclass linear model on one-hot codes, so we warm start from the boosted values and run Adam on cross entropy with a penalty toward the boosted solution, early-stopping on a separate tune slice. Three parameterisations are compared: one gain per round (`round`), one per (round, class) (`round_class`), and the full table (`table`).

Mass-adaptive leaf smoothing. Most buckets are empty (about 203 of 256 on Pedestrian), and an empty bucket contributes nothing. `shrinkage_tau` mixes each bucket's statistics with the mean of its b Hamming neighbours, which differ from it by exactly one predicate, with weight $\alpha_s = \tau/(\tau + \sum_k H_r[s, k])$. An empty bucket takes its neighbours' evidence in full, while a well-populated one is left almost unchanged.

3.5 Implementation

HashBoost runs entirely on the GPU in PyTorch. Three layout decisions carry most of the performance:

- **Codes are round-major and one byte each.** A batch's codes are a `uint8` tensor of shape (R, n), built one bit at a time so the largest intermediate is a single gather. With `torch.compile`, each bit becomes one fused kernel.
- **Gradient and hessian are fused.** G and H share one $(R, 2^b, 2k)$ tensor, so a single `scatter_add_` over an expanded view updates both, without materialising the per-round copy.
- **Work is chunked over rounds** to a fixed memory budget, so peak memory does not grow with R beyond the tables themselves.

Table 1: Kernel changes on Pedestrian, 800 rounds (`baseline`) and 1600 rounds (`capacity_2`). Results are identical. Compilation costs about 5.5 s on first use.

variant	before	eager	compiled	peak GPU (MB)
`baseline`	11.4 s	8.9 s	∼7.5 s	428 → 284
`capacity_2`	38.5 s	26.7 s	24.9 s	771 → 491

Table 2: Datasets. "Pool" is the number of rows outside test fold 0.

dataset	series	pool	channels	length	classes	QUANT features
Pedestrian	189,621	151,696	1	24	82	212
InsectSound	50,000	40,000	1	600	10	5,470
LenDB	1,244,942	983,483	3	540	2	14,940
Tiselac	99,687	79,907	10	23	9	2,040
Traffic	1,460,968	1,168,774	1	24	7	212

The original implementation, a numba CPU loop, is kept as a reference. Given the same predicates, the two agree to 4×10^{-7} in logits with identical argmax, and a test suite enforces that the defaults stay equal to it. On Pedestrian (800 rounds, 50 epochs) the GPU version cut training from 101.5 s to 10.6 s. Later kernel changes reduced it to about 7.5 s, with identical results (Table 1).

4 Experimental Setup

Datasets. We use all 26 classification datasets in MONSTER [9] (Table 3). Development used five of them (Table 2). MONSTER supplies five fixed cross-validation folds. All development and tuning held out fold 0 and never looked at its test rows. The final benchmark (Section 5.1) holds out each fold in turn.

Splits. In development (Sections 5.2, 5.5 and 5.6), models are trained on 65,536 rows drawn from the pool by a fixed shuffle (seed 42). InsectSound uses 32,768, because its pool is too small for 65,536 plus the evaluation slices. The next 4,096 rows are the validation set and the 4,096 after that are a tune slice, used only for decisions made after training (early stopping of the readout). Streamed runs (Section 5.4) train on the whole pool, and hold the same validation rows as the 65,536-row runs they are compared to. Every comparison between HashBoost and a tree model on MONSTER uses these same training and validation rows.

The final benchmark (Section 5.1) trains on the whole pool. From the same seed-42 shuffle it takes a validation slice and a tune slice of 4,096 rows each (a fifth of the pool each where the pool has fewer than 73,728 rows), and trains on all the rest. On fold k the pool is everything outside test fold k, and the slices are drawn from it the same way.

The exploratory Pedestrian sweeps (Tables 7, 12 and 15) and the QUANT/PULSAR comparison were run earlier, on a different shuffle (seed 123). They are compared only with each other, except where Table 7 sets them beside XGBoost, and that one comparison crosses the two splits. With 4,096 validation rows, the binomial standard error at 20% error is $\sqrt{0.2 \cdot 0.8/4096} \approx 0.006$, so a gap under ∼0.01 across the splits is not a result.

UCR. We also use the 112 univariate UCR datasets [26] with equal lengths and no missing values, the set the bake-off reports on [27]. These are the archive's 128 datasets less the 15 with missing values or variable lengths, and less Fungi, which has one training series per class. Each dataset uses the archive's default train/test split. The training sets are small: they range from

Table 3: The 26 MONSTER datasets. "Pool" is the number of rows outside test fold 0, and "train" the rows the benchmark trains on after validation and tune slices are taken from the pool. Classes are those present in the labels.

dataset	series	pool	train	classes	QUANT features
AudioMNIST-DS	30,000	24,000	15,808	10	35,349
CornellWhaleChallenge	30,000	24,000	15,808	2	35,349
CrowdSourced	12,289	9,456	5,674	2	31,542
DREAMERA	170,246	140,638	132,446	2	31,542
DREAMERV	170,246	140,638	132,446	2	31,542
FordChallenge	36,257	29,003	20,811	2	14,130
FruitFlies	34,518	27,614	19,422	3	44,031
InsectSound	50,000	40,000	32,768	10	5,470
LakeIce	129,280	103,424	95,232	3	1,652
LenDB	1,244,942	983,483	975,291	2	14,940
MosquitoSound	279,566	223,652	215,460	6	33,178
Opportunity	17,386	12,184	7,312	5	117,746
PAMAP2	38,856	28,961	20,769	12	54,184
Pedestrian	189,621	151,696	143,504	82	212
S2Agri-10pc-17	5,850,881	4,661,858	4,653,666	17	2,120
S2Agri-10pc-34	5,850,881	4,661,858	4,653,666	29	2,120
STEW	28,512	21,384	13,192	2	31,542
Skoda	14,117	11,290	6,774	11	62,520
TimeSen2Crop	1,135,511	839,971	831,779	15	30,096
Tiselac	99,687	79,907	71,715	9	2,040
Traffic	1,460,968	1,168,774	1,160,582	7	212
UCIActivity	10,299	7,898	4,740	6	10,440
USCActivity	56,228	43,060	34,868	12	6,252
WISDM	17,166	11,989	7,195	6	3,126
WISDM2	149,034	98,094	89,902	6	3,126
WhaleSounds	105,163	84,130	75,938	8	22,105

16 to 8,926 series (median 190), 27 have 50 or fewer, and only 11 have 1,000 or more. Lengths range from 15 to 2,844 and class counts from 2 to 60. UCR has no validation split, so every model runs at fixed settings, nothing is chosen after training, and we report test error.

Features. QUANT [7] (depth 6, div = 4) is fitted on the first training batch, and is the transform used for every result unless stated otherwise. PULSAR [8] is supervised, and its Fisher-score selection is fitted on the training rows alone. Both are our PyTorch ports. The PULSAR port matches upstream's feature counts exactly and its global features to 10^{-4}. About 0.5% of its local features differ, all at ties.

Baselines. XGBoost (GPU `hist`, depth 6), LightGBM (31 leaves) and CatBoost (GPU, depth 6), all with learning rate 0.05, up to 1,000 rounds and early stopping after 50 rounds without improvement. A single unpruned decision tree is included as a floor. In these development runs the tree models' figures are the best point on their validation curves, while HashBoost's are its final error. That favours the trees, and we leave it as run, because the final benchmark (Section 5.1) supersedes these numbers. There every model stops on the tune slice and is scored on test folds it never saw.

In the final MONSTER benchmark the baselines are XGBoost, tuned by the same procedure as HashBoost (below), and ExtraTrees with QUANT's own settings (200 trees, entropy criterion, a tenth of the features per split), trained on the same rows. ExtraTrees chooses nothing after training, so it never uses the tune slice.

On UCR, where there is nothing to early-stop on, the baselines are ExtraTrees with QUANT's own settings (200 trees, entropy criterion, a tenth of the features per split) [7], on the same QUANT features as HashBoost, and XGBoost at its library defaults (100 rounds, learning rate 0.3, depth 6).

HashBoost defaults. $b = 8$, $\eta = 0.1$, batch size 4,096, 50 epochs. For 65,536 rows that is 16 batches per epoch and 800 rounds. Most UCR training sets fit in a single batch, where 50 epochs would mean 50 rounds. On UCR the budget is therefore 800 rounds of the same model: batches of at most 4,096 rows, cycled until 800 rounds have been added.

Configurations for the MONSTER benchmark. Both boosters were tuned the same way, on fold 0's validation error only. We screened one change at a time on 10 of the 26 datasets (CrowdSourced, DREAMERA, FordChallenge, InsectSound, LenDB, PAMAP2, STEW, Traffic, UCIActivity and WISDM), fixed two configurations per model, and ran both on all 26. For each dataset, the configuration with the lower fold-0 tune error was then frozen. Each frozen configuration was then trained once on each of the five folds and scored once on that fold's test rows, at the round its tune error chose.

HashBoost's two configurations both train for up to 5,000 batches of 4,096 rows, evaluate every 50 batches, stop once their best tune evaluation is 1,000 batches old, and freeze rounds older than 128 batches:

- `G` draws each of a round's 6 bits as the best of 64 hard-pair candidates by boosting gain (`GainSplitter`), with a leaf penalty $\epsilon = 100$ and 8 rounds per batch. It was picked on 18 datasets.
- `L` uses the default 8-bit hard-pair rounds with $\epsilon = 1$ and 16 rounds per batch. It was picked on the 8 where gain selection overfits: the largest pools (LenDB, S2Agri, Traffic, Pedestrian), WhaleSounds and the EEG sets DREAMERA and DREAMERV.

The pick was the same for all five model seeds on 25 of the 26 datasets. XGBoost's screen raised its cap to 5,000 rounds with 100 rounds of patience, then varied depth, learning rate, subsampling, L2 regularisation and the number of bins. Its two configurations are `d4` (depth 4, learning rate 0.1, picked on 17 datasets) and `d8` (depth 8, learning rate 0.05, 64 bins, picked on 9). At 256 bins, depth 8 ran out of the GPU's 46 GB on several datasets.

Hardware. One NVIDIA RTX 4060 (8 GB) with 7.9 GB of host RAM under WSL2. The memory limits shaped much of the design. For example, a 3,200-round single model peaked near the card's limit. The final MONSTER benchmark ran on Monash's M3 cluster: each boosting run alone on one NVIDIA L40S (46 GB), and ExtraTrees on CPU nodes with 36 cores and 200 GB of RAM.

Measurement protocol. The final MONSTER benchmark reports test misclassification error, averaged over the five folds, with one run per fold. In development we report validation misclassification error. Every variant is compared with a baseline rerun *in the same sweep*. Differences under about 0.006 on Pedestrian are not treated as results, for the reasons in Section 5.6. Where a change does not alter training (the readout), the comparison is paired against the same fitted model. Unless stated, results are the mean ± standard deviation over 3 runs.

Table 4: MONSTER: test error, mean of the five test folds, one run per fold. HashBoost and XGBoost use the configuration each picked on fold 0 (Section 4). Times are training seconds summed over the five folds, with QUANT included: HashBoost and XGBoost on one L40S, ExtraTrees on 36 CPU cores. The lowest error in each row is bold. †One fold rerun with a fixed intercept (see text).

	HashBoost		XGBoost		ExtraTrees	time (s)		
dataset	config	error	config	error	error	HB	XGB	ET
AudioMNIST-DS	G	**0.0816**	d4	0.0884	0.1270	503	885	429
CornellWhaleChallenge	G	0.1250	d8	**0.1233**	0.1262	290	424	412
CrowdSourced	G	0.3501	d4	0.3622	**0.3396**	192	95	119
DREAMERA	L	0.5161	d8	0.5312	**0.4867**	438	1,775	4,253
DREAMERV	L	**0.4036**	d8	0.4038	0.4179	434	2,264	4,235
FordChallenge	G	0.0823	d8	0.0731	**0.0715**	196	152	224
FruitFlies	G	**0.0458**	d4	0.0462	0.0619	383	384	670
InsectSound	G	**0.1872**	d4	0.1891	0.2268	520	357	148
LakeIce	G	**0.0018**	d4	0.0019	0.0024	195	38	116
LenDB	L	0.0538	d4	**0.0522**	0.0551	728	1,364	25,843
MosquitoSound	G	0.1052	d4	**0.1008**	0.1596	1,855	5,054	7,366
Opportunity	G	0.1322	d4	**0.1305**	0.1394	399	852	761
PAMAP2	G	**0.2143**	d4	0.2246	0.2288	277	1,025	1,013
Pedestrian	L	**0.1746**	d8	0.1772	0.2135	2,588	792	61
S2Agri-10pc-17	L	0.2829	d8	0.2525	**0.2521**	1,463	30,068	18,291
S2Agri-10pc-34	L	0.2828	d8	**0.2530**	0.2638	2,013	61,933	16,151
STEW	G	**0.1937**	d4	0.1993	0.2371	325	310	382
Skoda	G	0.0567	d4	0.0610	**0.0565**	354	784	293
TimeSen2Crop	G	0.1412	d4	**0.0849**	0.1808	1,926	35,655	32,910
Tiselac	G	**0.2139**	d4	0.2140	0.2172	396	268	115
Traffic	L	0.2841	d8	0.2939	**0.2488**	846	407	375
UCIActivity	G	**0.0521**	d4	0.0675	0.0855	169	46	21
USCActivity	G	0.2428	d4	**0.2396**	0.2751	497	305	188
WISDM	G	0.1152	d4	**0.1131**†	0.1298	161	22	17
WISDM2	G	0.2556	d4	**0.2541**	0.2614	458	206	297
WhaleSounds	L	0.3128	d8	**0.2802**	0.2936	612	3,633	1,856
total						18,214	149,099	116,546

5 Results

5.1 MONSTER: the five test folds (RQ1)

Table 4 gives each model's test error on all 26 MONSTER datasets, averaged over the five test folds. Table 5 summarises the pairwise comparisons, and Figure 1 plots them. Every run finished. Each restored model reproduced the validation error it had recorded at its tune-chosen round, which checks that test error was scored at that point.

HashBoost and tuned XGBoost are level. HashBoost's error is 1.016 times XGBoost's in geometric mean. It has the lower error on 14 datasets and the higher on 12, and on 61 of the 130 dataset-folds. The Wilcoxon test finds no difference ($p = 0.98$), and the validation results on fold 0 agreed (ratio 1.047, $p = 0.20$). Where one model clearly leads, it is usually XGBoost. It does so on six datasets: TimeSen2Crop (HashBoost's error is 1.66 times XGBoost's), FordChallenge, WhaleSounds, both S2Agri subsets (1.12–1.13) and MosquitoSound (1.04). HashBoost clearly leads only on AudioMNIST-DS (0.92). Its leads on UCIActivity, PAMAP2 and CrowdSourced are too variable across folds to be clear. The tuning datasets favour HashBoost: it is 0.973 times XGBoost's error on the 10 it was screened on ($p = 0.064$, unadjusted) and 1.045 times on the

Table 5: MONSTER: pairwise comparisons over the 26 datasets. The ratio is the geometric mean of the first model's error over the second's. W/T/L counts the datasets where the first has the lower, equal and higher five-fold mean. "Clear" counts those where a 95% paired-t interval over the five per-fold differences excludes zero, either way. p is a two-sided Wilcoxon signed-rank test on the five-fold means, Holm-adjusted over the three comparisons.

comparison	ratio	W/T/L	clear (W/L)	p
HashBoost vs XGBoost	1.016	14/0/12	1/6	0.98
HashBoost vs ExtraTrees	0.904	18/0/8	9/5	0.11
XGBoost vs ExtraTrees	0.890	20/0/6	13/3	0.031

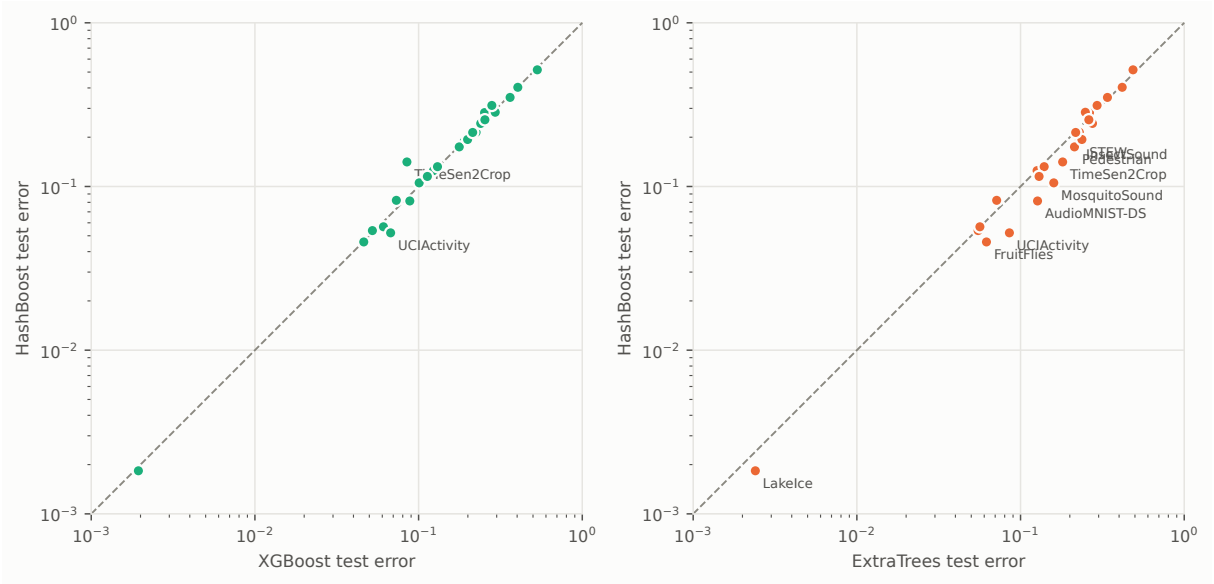


Figure 1: MONSTER: HashBoost's test error against each baseline's, mean of the five folds, one point per dataset. Below the diagonal, HashBoost has the lower error. Labelled points differ by more than 20%.

other 16 ($p = 0.16$). Both models were screened on those 10, but HashBoost was developed on them over many more rounds.

Both boosters beat ExtraTrees on MONSTER. The mean ranks are 1.77 for HashBoost, 1.77 for XGBoost and 2.46 for ExtraTrees (Figure 2). A Friedman test rejects equal ranks ($p = 0.016$). ExtraTrees is 0.69 behind both boosters, just beyond the Nemenyi critical difference of 0.65 at $\alpha = 0.05$. On UCR it was significantly ahead of both (Section 5.3). Its largest losses are on the big acoustic and sensor pools: MosquitoSound (1.52 times HashBoost's error), AudioMNIST-DS, FruitFlies, Pedestrian, InsectSound and TimeSen2Crop. It leads both boosters clearly only on Traffic (0.88 times HashBoost's error), where its validation error is also the lowest. UCR's training sets have a median of 190 series and MONSTER's pools have thousands to millions, which is consistent with averaging unpruned random trees suiting small training sets and boosting suiting large ones.

Cost. On the same GPU, HashBoost trained 8.2 times faster than XGBoost in total: 18,214 s against 149,099 s over the 130 dataset-folds. Without S2Agri, where XGBoost spent 92,001 s, the factor is 3.9 (14,738 s against 57,098 s). On small pools XGBoost is faster, because HashBoost's cost per round is mostly fixed: UCIActivity took 169 s against 46 s, and LakeIce 195 s against 38 s. ExtraTrees took 116,546 s on 36 CPU cores, which is not directly comparable.

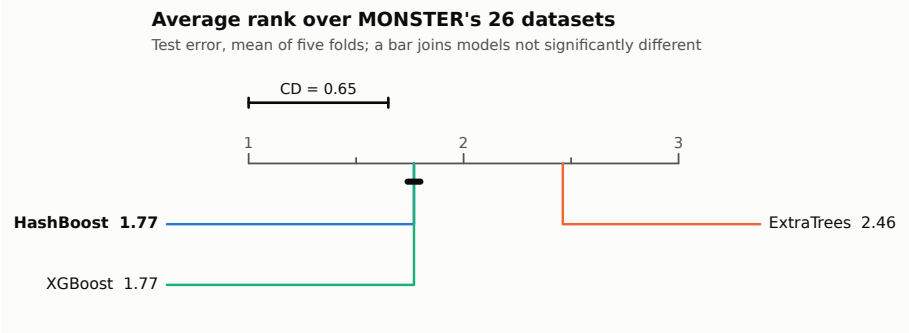


Figure 2: Critical difference diagram over MONSTER's 26 datasets [28]: test error, mean of the five folds. A Friedman test rejects equal ranks ($p = 0.016$). The bar joins models within the Nemenyi critical difference (0.65) of each other at $\alpha = 0.05$.

Figure 3 shows where the time goes on the six largest pools. On S2Agri-10pc-34, HashBoost reaches its tune-chosen point (validation 0.0200) after 375 s, and XGBoost reaches its point (0.0166) after 12,661 s. On TimeSen2Crop the times are 398 s and 5,256 s, for 0.0278 against 0.0227. On these pools XGBoost's extra hour or more buys its accuracy lead. HashBoost, though, stopped at its budget of 5,000 batches on every S2Agri and Traffic fold but one, with its best tune error in the last 1,000 batches. It was still improving there, so its error on these pools is partly set by the budget. On Traffic and Pedestrian its curve sits at or below XGBoost's throughout. On LenDB it reaches 0.0410 in 69 s, where XGBoost needs 292 s to reach 0.0398.

Test error is about 3.3 times validation error. That holds for all three models (geometric means of 3.33 for HashBoost, 3.23 for XGBoost and 3.35 for ExtraTrees). The validation and tune slices are random draws from the pool, but the test folds differ from it in distribution. On DREAMERA, XGBoost's validation error is half HashBoost's (0.071 against 0.135), yet both are near 0.5 on test, with HashBoost slightly lower. Every model is tuned on the same optimistic slices, so the comparison stays fair, but validation gaps on MONSTER do not translate directly into test gaps. Nine datasets keep test error close to validation: CornellWhaleChallenge, FruitFlies, InsectSound, LakeIce, MosquitoSound, Pedestrian, Skoda, Traffic and WhaleSounds.

A failure in XGBoost's intercept. WISDM's test fold 3 holds every series of two of its six classes, so that fold's training pool lacks them. XGBoost 3.4 estimates each class's starting score from the training labels, and here it produced +3.4 and +4.8 for the two absent classes and −8.1 for the two commonest. Training error then swung between 0.57 and 0.91 from the first round and never fell, giving a test error of 0.683. Fixing the intercept at 0.5, XGBoost's default before version 2.0, gives 0.178 against HashBoost's 0.174. Table 4 uses the repaired run. As run, the HashBoost–XGBoost ratio is 0.992 ($p = 0.71$). No other run showed the pattern. HashBoost starts every class at a uniform prior and ExtraTrees predicts only the classes it was trained on, so neither is affected.

5.2 Development on five datasets

The results in this subsection come from development, before the benchmark above. They are validation error on fold 0 at 65,536 training rows. Table 6 compares *default* HashBoost (a single model, 50 epochs: 800 rounds, or 400 on InsectSound's 32,768 rows) with the tree ensembles, all on the same rows. HashBoost trails the best tree model on every dataset. The gap ranges from 0.013 on LenDB to 0.083 on InsectSound. HashBoost fits in 3–14 s on the GPU. Pedestrian ran first, so its figure includes a one-off compilation of about 5 s. XGBoost took 333 s on Pedestrian. Among the trees, the winner changes: XGBoost is best on Pedestrian, InsectSound and LenDB,

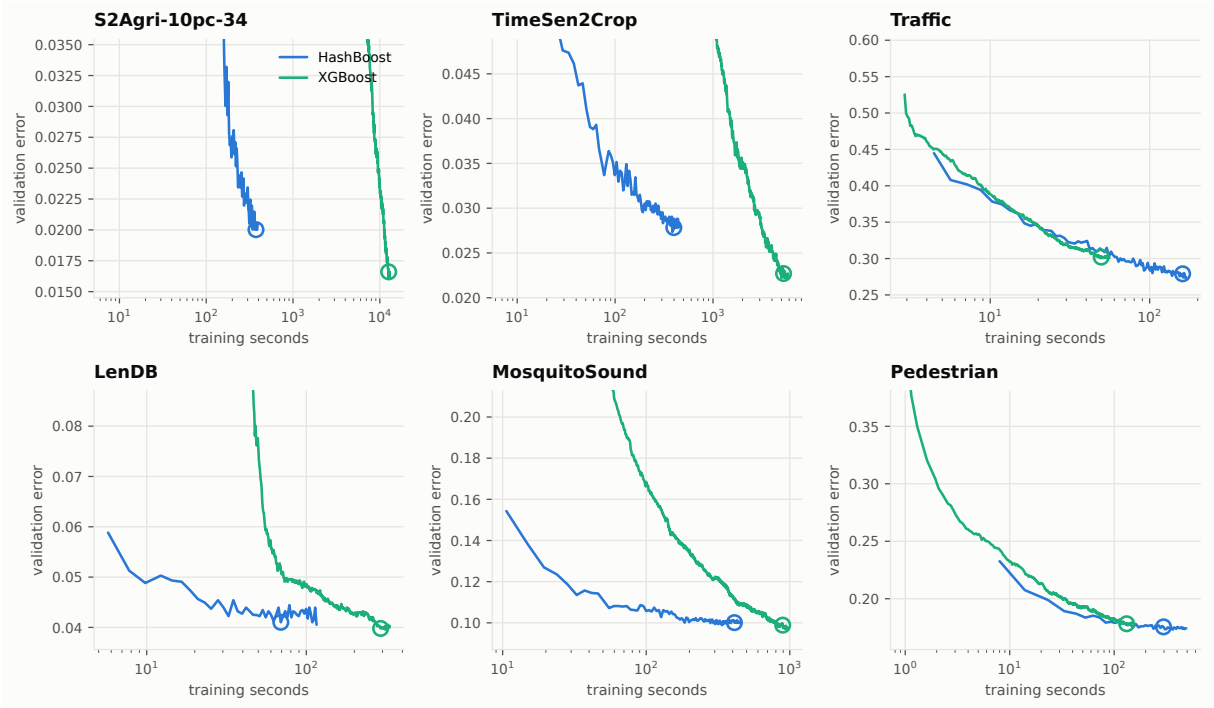


Figure 3: Validation error against training time on fold 0 for the six largest pools, each booster at its frozen configuration, on one L40S. QUANT is included. HashBoost's time is measured at each evaluation; XGBoost's is its build plus its mean time per round, since its rounds cost nearly the same each. Rings mark the tune-chosen point. The horizontal axis is logarithmic.

and LightGBM on Tiselac and Traffic. CatBoost, whose oblivious trees are structurally closest to HashBoost, is worse than default HashBoost on Pedestrian and Tiselac. On Traffic it lies between HashBoost and the other two tree models.

These are default settings on one split, with the trees selected on validation. Section 5.1 replaces them with tuned configurations of both boosters, whole training pools and test folds. The gap here closed mainly through capacity (more rounds per batch), the full budget, and the regularised gain-selected rounds of `G`.

Closing the gap on Pedestrian. Pedestrian received the most tuning. Table 7 shows the path from the default to XGBoost's level. Two things stand out. Capacity helps more than averaging at equal hash count (`capacity_2` 0.2150 against two bagged 800-round models 0.2204). But bagging is far cheaper per hash, and combining the two is best. At 3,200 hashes, four 800-round models take 43.5 s where one 3,200-round model takes 818 s. `bagged_4_capacity_2` matches XGBoost at less than half its training time. Refitting its readout jointly takes it to 0.2006, the first HashBoost result below XGBoost's 0.2041. These sweeps used the earlier split, however, so that comparison crosses splits and a gap of 0.0035 is not a result (Section 4).

Feature transform. The largest single gain we measured came from the features, not the model. Replacing QUANT with PULSAR reduced InsectSound's error from 0.2699 ± 0.0094 to 0.2262 ± 0.0044 (3 runs, same rows), about half the gap to XGBoost on QUANT. It *increased* Pedestrian's from 0.2326 ± 0.0059 to 0.2481 ± 0.0034. On whole-pool Traffic the two were level (0.3895 against 0.3931, one run each), with PULSAR taking twice the wall time. One untested explanation for Pedestrian is dilution. HashBoost draws features uniformly, and PULSAR offers 3,000 columns where QUANT offers 212. [**TODO:** XGBoost on PULSAR features, to separate the transform's effect from its interaction with HashBoost.]

Table 6: Validation error, 65,536 training rows (InsectSound 32,768), QUANT features, one run each, all on the seed-42 split. HashBoost is the default single model at its final round. The trees are at the best point on their validation curves. LightGBM diverged on Pedestrian (final error 0.896). The gap is to the best tree model. –: not run. An earlier run of default HashBoost on the same Pedestrian rows read 0.2253.

dataset	HashBoost	fit	XGBoost	LightGBM	CatBoost	tree	gap
Pedestrian	0.2356	13.9 s	**0.2041**	0.2849*	0.2427	0.4397	+0.032
InsectSound	0.2712	4.0 s	**0.1887**	–	–	–	+0.083
LenDB	0.0591	8.6 s	**0.0459**	–	–	–	+0.013
Tiselac	0.0818	4.0 s	0.0659	**0.0632**	0.0928	0.1650	+0.019
Traffic	0.4365	3.4 s	0.3989	**0.3936**	0.4226	0.5310	+0.043

Table 7: Pedestrian, seed-123 split, 3 runs each. Rows come from different sweeps, whose baselines ranged from 0.2214 to 0.2270 (Section 5.6), so differences between adjacent rows under ∼0.006 are not meaningful. The XGBoost figure is from the seed-42 split, so that comparison crosses splits (Section 4).

model	hashes	val error	wall
`baseline`	800	0.2240 ± 0.0004	10.5 s
`bagged_4` (4 × 800)	3,200	0.2128 ± 0.0024	43.5 s
`capacity_4` (1 × 3,200)	3,200	0.2103 ± 0.0012	818 s
`bagged_4_capacity_2` (4 × 1,600)	6,400	0.2059 ± 0.0017	148 s
+ `round_class` readout	6,400	$\mathbf{0.2006} \pm 0.0025$	375 s
XGBoost (best point on curve)		0.2041	333 s

5.3 The UCR archive (RQ3)

Table 8 summarises default HashBoost, ExtraTrees and XGBoost on the UCR 112, each dataset on its default split and run once. First, a check on the pipeline: our ExtraTrees reproduces the published QUANT results on the same splits [27]. Its mean error is 0.1475 against 0.1456, the correlation across datasets is 0.996, and the 12 datasets that differ by more than 0.02 split six each way.

A Friedman test rejects equal performance across the three models ($p = 3.6 \times 10^{-22}$), and the Nemenyi test splits them into two groups (Figure 4). ExtraTrees is significantly better than both boosters. HashBoost and XGBoost are not significantly different: their mean ranks are 2.36 and 2.39, and HashBoost has the lower error on 58 datasets and the higher on 51. Against ExtraTrees, HashBoost has the lower error on only 8.

The record against XGBoost depends on the size of the training set (Table 9). HashBoost's lead over XGBoost comes entirely from the 58 datasets with at most 200 training rows, where XGBoost's defaults do badly. There HashBoost wins 39 and loses 17. Above 200 rows it wins 19 and loses 34. On the 11 datasets with at least 1,000 rows it trails both baselines (Holm-adjusted p of 0.012 against ExtraTrees and 0.032 against XGBoost). Against ExtraTrees it loses in every band (Figure 5). The gap to ExtraTrees correlates more with the number of classes ($r = 0.31$) than with training size ($r = -0.11$). Three of the five widest gaps are the Pig datasets, which have 52 classes and two training series per class.

Cost follows from the fixed budget. HashBoost fits every dataset in 1.4–2.8 s (median 1.6 s). A training set that fits in one batch is re-accumulated every round, so the time hardly depends on the data. ExtraTrees fits most small datasets in 0.2 s on the CPU. On the 11 largest datasets HashBoost is the fastest in total (24 s, against 37 s for ExtraTrees and 101 s for XGBoost), but

Table 8: UCR 112: test error on the default splits, one run per dataset. Mean rank is among the three models we ran (1 is the lowest error). W/T/L is HashBoost's record against each row: the datasets where its error is lower, equal and higher. Total fit is summed over the 112. p is a two-sided Wilcoxon signed-rank test against HashBoost, Holm-adjusted over the four comparisons. The published rows are the bake-off's resample 0, which is the default split, and are shown for reference.

model	mean error	median	mean rank	W/T/L	p	total fit
HashBoost	0.1941	0.1705	2.36	–	–	183 s
ExtraTrees	**0.1475**	0.1205	**1.25**	8/8/96	2.6×10^{-14}	70 s
XGBoost	0.2101	0.1912	2.39	58/3/51	0.082	413 s
QUANT (published)	0.1456	0.1177		13/5/94	4.9×10^{-14}	
HC2 (published)	0.1236	0.0753		8/7/97	3.8×10^{-15}	

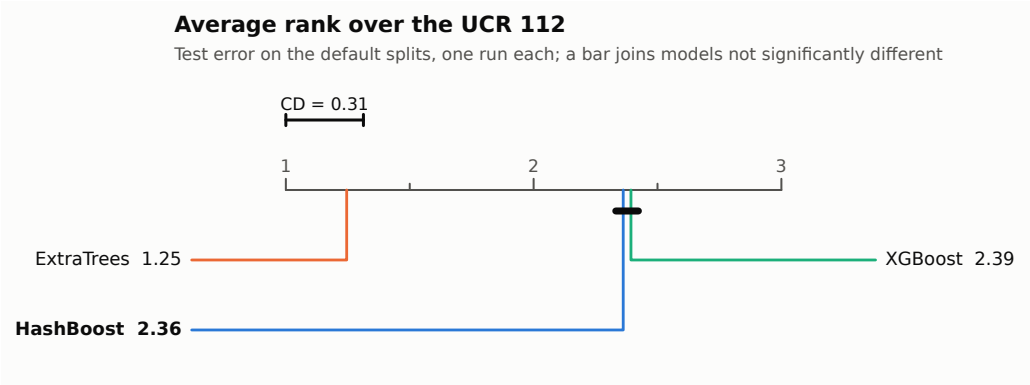


Figure 4: Critical difference diagram over the UCR 112 [28]. A Friedman test rejects equal ranks ($p = 3.6 \times 10^{-22}$). A bar joins models that are within the Nemenyi critical difference (0.31) of each other at $\alpha = 0.05$.

it is also the least accurate there. The budget is not a ceiling. HashBoost reaches zero training error on 111 of the 112 datasets, yet its mean test error keeps falling, from 0.202 at 400 rounds to 0.194 at 800. This was recorded for plotting only, and nothing was chosen on it.

Bits and rounds. The default bit width and budget were set on MONSTER, where each of a round's 256 buckets sees thousands of rows. Most UCR training sets cannot fill them. Table 10 varies both, with one run per cell and the default rerun in the same sweep as the control. The rerun moved the mean error by 0.0015 (0.1941 to 0.1956), with no systematic direction (Wilcoxon $p = 0.21$). Every other cell beats the control, with Holm-adjusted Wilcoxon $p \leq 5.2 \times 10^{-7}$. Fewer bits help at both budgets. More rounds help at every width, and most at two bits, where each round is the weakest learner. The best width grows with the training set (Figure 6), as the bucket argument predicts: at 3,200 rounds two bits is best below 1,000 training rows and four bits above. The gain from fewer bits is larger where there are fewer training rows per class ($r = -0.34$ against $\log n/k$).

A grid scored on test sets cannot also choose its own winner, so we choose a cell by two-fold cross-validation over datasets. The cell is picked on a random half of the datasets and scored on the other half, both ways round. Both halves choose two bits and 3,200 rounds. Chosen that way, HashBoost is significantly ahead of default XGBoost: its error is lower on 82 datasets and higher on 29 ($p = 4.5 \times 10^{-8}$). It is still behind ExtraTrees, lower on 28 and higher on 63 ($p = 1.2 \times 10^{-5}$), but the gap in mean error falls from 0.048 to 0.017. The bits are free. On sets this small, fit time is per-round overhead rather than arithmetic, and the 800-round cells take 179–188 s over all 112 datasets whatever the width. The extra rounds are not: the 3,200-round

Table 9: UCR 112 by training-set size: HashBoost's W/T/L against each baseline, and each model's mean test error.

training rows	datasets	vs ExtraTrees	vs XGBoost	HashBoost	ExtraTrees	XGBoost
≤ 50	27	3/0/24	19/1/7	0.1141	0.0646	0.1841
51–200	31	2/4/25	20/1/10	0.1981	0.1404	0.2198
201–999	43	2/4/37	16/1/26	0.2598	0.2176	0.2461
≥ 1000	11	1/0/10	3/0/8	0.1225	0.0973	0.1054

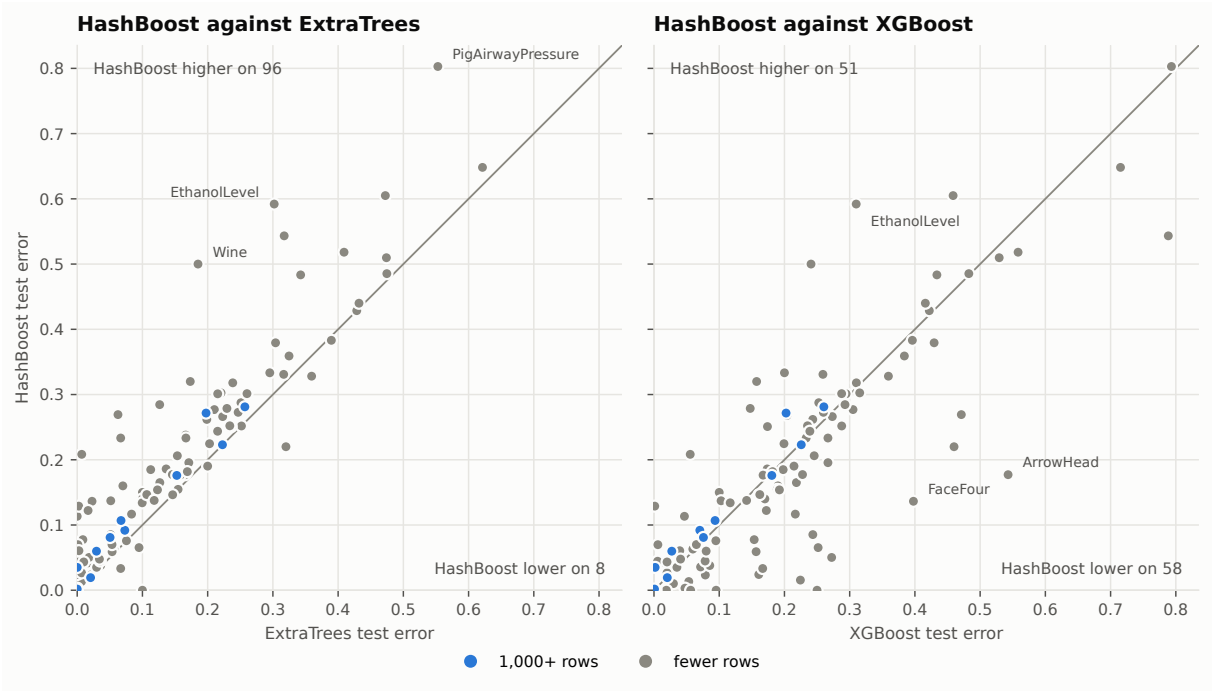


Figure 5: HashBoost's test error against each baseline's, one point per UCR dataset. Below the diagonal, HashBoost has the lower error. Blue marks the 11 datasets with at least 1,000 training rows.

cells take 7.5 times as long, against 70 s for ExtraTrees on the CPU.

5.4 Training cost and scaling with instances (RQ2)

Cost is quadratic in rounds. Every batch updates every round, so per-batch cost grows linearly with the number of rounds. On a streamed Traffic run it rose from 2 ms to 14.5 ms per batch over 2,840 rounds. Total training cost is therefore quadratic: 1,600 rounds took 37.5 s against 10.5 s for 800. Where the time goes depends on the dataset (Table 11). Many classes make the scatter-add dominate, and many features make encoding dominate.

Freezing makes it linear. Table 12 shows round freezing on Pedestrian. A 10-epoch window reproduces `capacity_2`'s accuracy in 2.4× less time. A second sweep repeated this (0.2129 in 10.3 s against 0.2137 in 24.7 s). The window is a real hyperparameter. On Pedestrian a 2-epoch window cost between +0.008 and +0.027 across four sweeps. On InsectSound even 2 epochs cost nothing measurable. On streamed whole-pool runs a 2-epoch window also cost nothing measurable, on Traffic (0.3877 against 0.3922) and on LenDB (0.0751 against 0.0745). Freezing held Traffic's per-batch model cost near 7.5 ms instead of letting it grow.

Table 10: UCR 112: HashBoost's mean test error by bits per round and round budget, one run per cell. The default (8 bits, 800 rounds) was rerun in the same sweep. It is a sensitivity study on the test sets, so the best cell's own figure is optimistic; see the text for a choice made without reading the scored test sets.

bits	800 rounds	3,200 rounds
8 (default)	0.1956 (control)	0.1885
6	0.1844	0.1808
4	0.1767	0.1712
2	0.1750	**0.1647**

Table 11: Share of one training batch by stage.

	Pedestrian	InsectSound	LenDB
features / classes	212 / 82	5,470 / 10	14,940 / 2
ms per batch (at rounds)	27.9 (800)	13.5 (800)	18.3 (1,200)
accumulate	57%	21%	8%
encode	19%	43%	47%
predict	10%	11%	10%
refresh leaves	10%	3%	1%
transpose	1%	14%	28%
objective + propose	4%	9%	7%

Memory does not grow with the training set. Table 13 streams the whole LenDB training pool, 975,291 rows, and compares it with the 65,536-row runs on the same validation rows. The pool is 6.3 GB as raw series, more than the host can hold alongside anything else, and 58 GB after QUANT. Neither model materialises the features. HashBoost's peak GPU memory was *identical* at both sizes, 2,572 MB, because each batch is folded into the tables and freed. XGBoost's binned matrix (the ellpack) is almost exactly one byte per element and grows linearly with rows (Table 14). At the full pool it pages 14.6 GB from disk every round, at 171× the in-memory cost per round. Reaching the 179 rounds of the 65,536-row run would have taken about six hours.

More data needs more rounds. At the same number of rounds, the 65,536-row model is *better* than the full-pool model (0.0532 against 0.0691). The training errors explain it. The small model has memorised its training set (0.0049), while the full-pool model sits at 0.0608 against 0.0691 on validation, which is underfitting. About 1,200 rounds are enough to memorise 65,536 rows and nowhere near enough for 975,291, and the full-pool curve was still descending when the run stopped. HashBoost's scaling is limited by compute in rounds, not by memory, and the quadratic cost is what makes rounds expensive.

On Traffic, where the full pool is 1.16 million rows, whole-pool training does pay. From 65,536 rows HashBoost scored 0.4404 against XGBoost's 0.3997 and LightGBM's 0.3950. Ten streamed epochs over the whole pool (2,840 rounds) reached 0.3922 unfrozen and 0.3877 with a 2-epoch window, level with the tree models trained on 65,536 rows, in 139–154 s. In these streamed runs the model was only 14–19% of wall time on Traffic and under 4% on LenDB. Most of the time went to the QUANT transform and to reading the memory-mapped input.

Given more rounds, the full pool pays on LenDB too. Configuration `L` (16 rounds per batch, a 128-batch window) reached a validation error of 0.0428 ± 0.0018 over five seeds on the whole pool. That beats its own 65,536-row run (0.0532) and whole-pool XGBoost at its defaults (0.0471). Its peak GPU memory was 3.4–4.9 GB across the five test folds, larger than the default's 2.6 GB

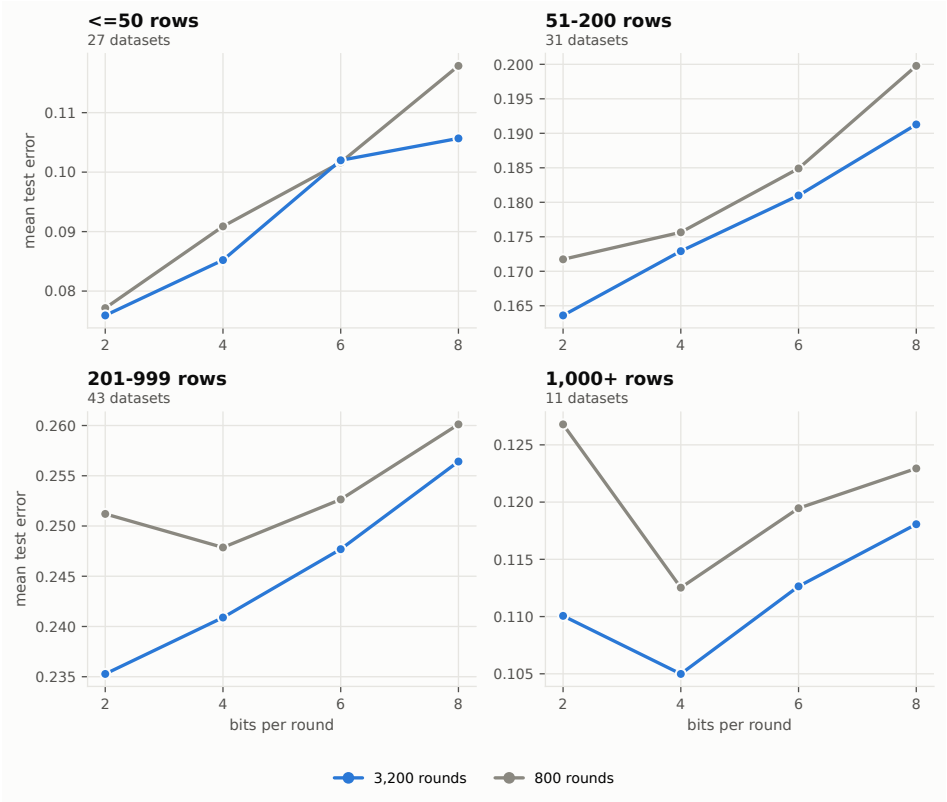


Figure 6: HashBoost's mean test error against bits per round, by training-set size, at 800 and 3,200 rounds. Each panel has its own vertical scale. The best width falls from four bits to two as the training sets shrink.

Table 12: Round freezing on Pedestrian, 3 runs, one sweep, compiled.

variant	val error	wall
`baseline` (800 rounds)	0.2252 ± 0.0023	8.4 s
`frozen_10ep`	0.2276 ± 0.0037	3.6 s
`frozen_2ep`	0.2332 ± 0.0042	1.8 s
`capacity_2` (1,600 rounds)	0.2133 ± 0.0025	25.6 s
`frozen_10ep_capacity_2`	$\mathbf{0.2135} \pm 0.0032$	**10.8 s**

because each batch adds 16 rounds. Tuned XGBoost peaked at 17.1–17.8 GB on the same pools. On the test folds the two are close (0.0538 against 0.0522).

5.5 What did not work

Smarter partitions make a worse ensemble. Choosing each round's predicates as the best of K candidates by boosting gain raised the number of occupied buckets from 79 to 119 of 256, and made the ensemble worse (0.2241–0.2339 against 0.2240 on Pedestrian). So did every other change that made individual rounds more informed or more expressive: richer QUANT features (div = 2: 0.2275; div = 1: 0.2434), row subsampling (0.2585 at half), statistic decay, leaf L2, oblique predicates (0.2277, consistently a little worse), and a hard top-25% feature filter (0.3420). The randomness of the predicates appears to be what makes the rounds diverse enough to add up, as in extremely randomised trees [16]. The exception came later, on whole MONSTER pools. There, gain selection over 64 candidates, combined with 6-bit rounds and a heavy leaf penalty ($\epsilon = 100$), was the largest single gain in the screen. It cut CrowdSourced's

Table 13: LenDB, streamed. One run each.

model	rows	rounds	train	val	wall	peak GPU
HashBoost	65,536	1,200	0.0049	0.0532	230 s	2,572 MB
HashBoost	975,291	1,195	0.0608	0.0691	560 s	2,572 MB
XGBoost	65,536	179	0.0236	**0.0459**	131 s	6,223 MB
XGBoost	975,291	3	0.0566	0.0627	702 s	6,021 MB

Table 14: XGBoost's external-memory ellpack on LenDB grows linearly with rows, and so does its per-round cost.

rows	ellpack	s/round
65,536	in RAM	0.68
122,880	1.8 GB	8.9
245,760	3.7 GB	16.2
975,291	14.6 GB	116.0

validation error from 0.042 to 0.006 and FordChallenge's from 0.105 to 0.062, though it raised DREAMERA's. This became configuration `G` (Section 4). The penalty appears to be what lets gain-chosen rounds generalise.

Feature weighting is dataset-specific. Drawing features in proportion to a mix of uniform and ANOVA-F scores reduced InsectSound's error by 0.027, but increased Traffic's by 0.011. Pedestrian, LenDB and Tiselac did not move. LenDB has the most features and was unaffected, so dilution by uninformative features is not the general explanation.

Other dead ends. Label smoothing (+0.007 and +0.033), hessian floors other than 10^{-3} (10^{-4}: +0.098), per-epoch reshuffling, uniformly drawn thresholds within the pair, and fixed neighbour shrinkage (two paired comparisons helped, two hurt, averaging zero). Heterogeneous bagging (members with different bit widths or partition families) reproducibly decorrelated the members, but only the axis-aligned/oblique mix improved accuracy, by 0.0027. That is inside the noise.

Small gains. Sampled hard pairs reduced Pedestrian's error in four sweeps out of four, by 0.001 to 0.006, and never hurt on the other four datasets. Mass-adaptive smoothing has a clean interior optimum at $\tau = 3000$ (Table 15). It gave +0.0047 on `capacity_2` in the same sweep, and gains less on top of bagging. Refitting the readout gives a paired +0.0078 on an 800-round model (`table`) and +0.0049 on the bagged ensemble (`round_class`). Rescaling whole rounds is worth nothing (+0.0000): the boosted leaves are wrong per class, not mis-weighted between rounds.

5.6 Run-to-run noise

Seeding does not make HashBoost reproducible. Float32 scatter-adds on the GPU accumulate in a nondeterministic order. That perturbs the logits, which reorders the cross-entropy ranking the predicates are drawn from, and every round after that differs. Identical configurations in three separate sweeps gave Pedestrian baselines of 0.2240, 0.2270 and 0.2214. That is a spread of 0.0056, with within-sweep standard deviations from 0.0004 to 0.0090.

The leaf dynamics amplify the perturbation. Under deterministic kernels, a 10^{-6} change to one round's statistics grew to 60 logits within 4 epochs. The amplifier is extreme leaves: a

Table 15: Mass-adaptive smoothing on Pedestrian, 800 rounds, 3 runs. Same sweep baseline: 0.2214 ± 0.0011. Typical bucket mass is about 10^4.

τ	100	300	1000	**3000**	10^4	$3 \cdot 10^4$	10^5	10^6
error	0.2197	0.2179	0.2178	**0.2149**	0.2199	0.2261	0.2467	0.3803

bucket holding a single rare-class example gets a Newton step of roughly $\eta/(\text{count} \cdot 10^{-3})$, and leaves reach ± 100. Clipping leaves to ± 3 holds the perturbation at 10^{-6} at no cost in accuracy. Same-seed runs still diverge once predicate selection meets a floating-point near-tie. InsectSound showed no such amplification.

This has two consequences for method. Comparisons must rerun their control in the same sweep, and unpaired differences under about 0.006 cannot be interpreted. Where a change does not alter training, paired comparison against the same fitted model removes the noise almost entirely: the `table` readout's standard deviation was 0.0002 paired against 0.0084 unpaired, on the same three runs.

6 Discussion

Can the tree be replaced? (RQ1, RQ3) Against boosted trees at their defaults, yes. Across the UCR 112, default HashBoost is statistically level with default XGBoost, and it is clearly ahead on training sets of a few hundred rows or fewer. It is not level with the strongest classifier for the same features: QUANT's ExtraTrees ranks significantly higher and has the lower error on 96 of the 112 datasets. On MONSTER, the picture changes. With both boosters tuned the same way and scored on the five test folds, HashBoost is level with XGBoost across all 26 datasets ($p = 0.98$) and trains 8.2 times faster. Both rank significantly ahead of ExtraTrees. Which classifier suits QUANT's features therefore depends on scale: on small training sets an average of random trees wins, and on large ones a booster wins, whether its weak learner is a tree or a global partition. On UCR, two settings close most of the gap to ExtraTrees: fewer bits, so that small training sets fill their buckets, and more rounds. With two bits and 3,200 rounds, chosen by cross-validation over datasets, HashBoost is significantly ahead of XGBoost and 0.017 behind ExtraTrees in mean error, though still significantly behind it. On Pedestrian, a bagged ensemble with a refitted readout reached XGBoost's validation error, though only on the earlier split. So the tree can be replaced, and on MONSTER, given the same tuning effort as XGBoost, at no cost in accuracy.

Why does randomness work? The negative results point in one direction. Every change that made a single partition better informed made the ensemble worse. A random partition over a mini-batch is a poor weak learner, but its errors are weakly correlated with those of its neighbours. It keeps improving as later batches refine its leaves. There are also two regimes. Pedestrian, with 82 imbalanced classes, is limited by noisy leaf estimates: halving the batch size (twice the rounds) cost +0.021. InsectSound is limited by capacity, and the same change gained 0.015.

Training time and scaling (RQ2). HashBoost's memory use is independent of the training set size, which is what lets it use whole MONSTER pools on hardware where XGBoost has to page from disk. The binding constraint is instead the number of rounds. Bagging divides the quadratic cost, and freezing removes it. With both, the cost of a streamed run is dominated by the feature transform and I/O, not the model.

Limitations.

- The MONSTER benchmark is one run per test fold. Over five model seeds on fold 0, `auto(G,L)`'s standard deviation was 0.0002 to 0.0024 on most datasets. Both models' configurations were chosen on 10 of the 26 datasets, where HashBoost had also been developed for longer. It does better there (0.973 against 1.045 on the other 16). The UCR results are test error, but from the default split only, one run each, where the bake-off averages 30 resamples.
- HashBoost's budget of 5,000 batches stopped it while still improving on the largest pools (S2Agri and Traffic), where the tune slice would otherwise have chosen a later round. A budget scaled with the pool might narrow its deficit on S2Agri; it was not tried, to keep the configuration frozen.
- On MONSTER, validation error underestimates test error about 3.3-fold, because the validation and tune slices are random draws from a pool whose test fold differs from it. Choices made on validation, for both models, may not be the best on test.
- The Pedestrian exploration was run on an earlier split, so its comparison with XGBoost crosses splits (Section 4).
- On UCR every model runs at fixed settings, and the baselines are at library defaults, which suits ExtraTrees better than XGBoost. The bits-and-rounds grid is scored on test sets. The cross-validated choice over datasets never reads the test set it is scored on, but it still reads the other half's.
- The final MONSTER benchmark compares against XGBoost and ExtraTrees only. LightGBM and CatBoost appear only in the five-dataset development runs, at their defaults.
- Development timings come from one consumer GPU under WSL2, and the MONSTER benchmark's from an L40S. ExtraTrees ran on CPUs, so its time is not directly comparable.

Future work. On small datasets: a bit width chosen per dataset from its rows per class, rather than one width for all; cheaper rounds, since at UCR sizes a round's cost is overhead, which would make budgets past 3,200 affordable; one-bit rounds (stumps); and bagging, which paid on Pedestrian. More generally: feature weighting or a lower PULSAR `top_percent` for HashBoost; batching QUANT's per-interval quantile calls, now the dominant cost of a streamed run; and understanding the chaotic leaf dynamics, which would reduce the noise floor that limits every comparison here.

7 Conclusion

HashBoost replaces the decision tree in gradient boosting with a random global partition: b threshold predicates, drawn from pairs of hard examples, that index a table of Newton-step leaves refined by every later mini-batch. It does no split search and never holds the training set. At its defaults it is level with default XGBoost across the 112 UCR datasets. With fewer bits per round, so that small training sets fill their buckets, and more rounds, it is significantly ahead of XGBoost and within 0.017 of QUANT's ExtraTrees in mean error, though still significantly behind it. On all 26 MONSTER datasets, scored on the five test folds, it is level with XGBoost tuned by the same procedure, trains 8.2 times faster on the same GPU, and, like XGBoost, ranks significantly ahead of ExtraTrees. Its memory use does not grow with the training set, so it trains on whole pools of millions of rows. The cost that limits it is the number of rounds, which round freezing reduces from quadratic to linear.

A Remaining work before submission

Delete this section before submission.

1. **More runs on UCR.** The UCR 112 numbers are one run on the default split; the bake-off uses 30 resamples.
2. **UCR and MONSTER configurations.** UCR's best setting (two bits, 3,200 rounds) and MONSTER's (`G`/`L`) differ. Say whether one rule (e.g. a width from rows per class) covers both, or present them as separate regimes.
3. **Figures.** Per-batch cost vs. rounds, frozen and unfrozen; peak memory vs. rows for both models.
4. **Bibliography.** Check the entries added for this draft (bagging, random forests, extra trees, BDT, NODE, LSH, backfitting, Gumbel-top-k).

References

[1] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, pp. 3146–3154, 2017.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems 31*, pp. 6638–6648, 2018.

[5] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[6] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems 35: Datasets and Benchmarks Track*, 2022.

[7] A. Dempster, D. F. Schmidt, and G. I. Webb, "QUANT: A minimalist interval method for time series classification," *Data Mining and Knowledge Discovery*, 2024.

[8] N. Cabello and L. Kulik, "Pulsar: Advancing interval-based time series classification to state-of-the-art performance," in *2025 IEEE International Conference on Data Mining (ICDM)*, pp. 1085–1094, 2025.

[9] A. Dempster, N. M. Foumani, C. W. Tan, L. Miller, A. Mishra, M. Salehi, C. Pelletier, D. F. Schmidt, and G. I. Webb, "Monster: Monash scalable time series evaluation repository," 2025.

[10] J. H. Friedman, "Stochastic gradient boosting," *Computational Statistics & Data Analysis*, vol. 38, no. 4, pp. 367–378, 2002.

[11] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[12] Y. Lou and M. Obukhov, "BDT: Gradient boosted decision tables for high accuracy and scoring efficiency," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1893–1901, 2017.

[13] S. Popov, S. Morozov, and A. Babenko, "Neural oblivious decision ensembles for deep learning on tabular data," in *International Conference on Learning Representations*, 2020.

[14] A. Buja, T. Hastie, and R. Tibshirani, "Linear smoothers and additive models," *The Annals of Statistics*, vol. 17, no. 2, pp. 453–510, 1989.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[16] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.

[17] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pp. 604–613, 1998.

[18] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020.

[19] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. Bagnall, "HIVE-COTE 2.0: A new meta ensemble for time series classification," *Machine Learning*, vol. 110, no. 11, pp. 3211–3243, 2021.

[20] A. Dempster, D. F. Schmidt, and G. I. Webb, "MiniRocket: A very fast (almost) deterministic transform for time series classification," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, (New York), pp. 248–257, ACM, 2021.

[21] A. Dempster, D. F. Schmidt, and G. I. Webb, "Hydra: Competing convolutional kernels for fast and accurate time series classification," *Data Mining and Knowledge Discovery*, vol. 37, pp. 1779–1805, 2023.

[22] H. Deng, G. Runger, E. Tuv, and M. Vladimir, "A time series forest for classification and feature extraction," *Information Sciences*, vol. 239, pp. 142–153, 2013.

[23] M. Middlehurst, J. Large, and A. Bagnall, "The canonical interval forest (CIF) classifier for time series classification," in *2020 IEEE International Conference on Big Data (Big Data)*, pp. 188–195, 2020.

[24] N. Cabello, E. Naghizade, J. Qi, and L. Kulik, "Fast, accurate and interpretable time series classification through randomization," 2021.

[25] W. Kool, H. van Hoof, and M. Welling, "Stochastic beams and where to find them: The Gumbel-Top-k trick for sampling sequences without replacement," in *Proceedings of the 36th International Conference on Machine Learning*, pp. 3499–3508, 2019.

[26] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The UCR time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[27] M. Middlehurst, P. Schäfer, and A. Bagnall, "Bake off redux: A review and experimental evaluation of recent time series classification algorithms," *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 1958–2031, 2024.

[28] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.